Submission Template for

INFS2200/INFS7903 Assignment 2

Semester 2, 2026

Submission of Screenshot for Assignment 2

Submitted by:

Student Name	Student ID	Practical Session	Tutor Name
Yuyang Gong	49079984	Friday 9:00-10:00	Mr. Li, Jiayun

About AI Usage:

Did you use AI in this assignment?	Yes.
The purpose of AI used in this assignment?	To develop SQL solutions, run PostgreSQL experiments, interpret execution plans and transaction behaviour, and prepare the report.
To what extend that AI is used in this submitted work?	AI generated substantial parts of the SQL, local testing, explanations and document formatting. This is an AI-assisted draft for student review before submission.
Did you discuss your AI usage with the tutor during the "*Show to Tutor*" Lab sessions?	No. A2 has not yet been shown to the tutor and AI usage has not yet been discussed.

Screenshot Q1.1 (1)	Marks: /1

PostgreSQL execution transcript 1.1.1_1 | part 1/1

```
CREATE VIEW V_BUSIEST_BAYS AS
WITH collection_counts AS (
    SELECT m.body_type, s.station_id, s.station_name,
           COUNT(*) AS collections
    FROM BOOKING_EVENTS e
    JOIN VEHICLES v ON v.vehicle_id = e.vehicle_id
    JOIN MODELS m ON m.model_code = v.model_code
    JOIN STATIONS s ON s.station_id = v.station_id
    WHERE e.event_type = 'Collect'
    GROUP BY m.body_type, s.station_id, s.station_name
), ranked_stations AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY body_type
        ORDER BY collections DESC, station_id
    ) AS rn
    FROM collection_counts
)
SELECT body_type, station_name, collections
FROM ranked_stations
WHERE rn = 1
ORDER BY collections DESC, body_type;
CREATE VIEW
```

Source: 1.1.1_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The first CTE counts Collect events for each body type and station. ROW_NUMBER partitions those counts by body type and picks the largest count; station_id resolves ties deterministically so exactly one station is returned per represented body type. The final ORDER BY puts the largest winner first.

The schema records a station on VEHICLES, rather than on BOOKING_EVENTS. The station reported is therefore the vehicle's recorded station, which is the location available in the supplied schema.

Screenshot Q1.1(2)	Marks: /0.5

PostgreSQL execution transcript 1.1.2_1 | part 1/1

```
SELECT * FROM V_BUSIEST_BAYS;
  body_type   |  station_name  | collections
--------------+----------------+-------------
 Sedan        | Carindale Park |         973
 SUV          | Chermside Park |         838
 Hatchback    | Chermside Park |         794
 People Mover | St Lucia Hub   |         510
 Van          | Chermside Park |         384
 Ute          | Carindale Park |         333
(6 rows)
```

Source: 1.1.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.1.2_2 | part 1/3

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM V_BUSIEST_BAYS;
QUERY PLAN
-----------------------------------------------------------------------------------------
Sort  (cost=4782.24..4782.25 rows=2 width=32) (actual time=28.169..31.781 rows=6.00
  loops=1)
 Sort Key: ranked_stations.collections DESC, ranked_stations.body_type
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1665
 ->  Subquery Scan on ranked_stations  (cost=4682.66..4782.23 rows=2 width=32) (actual
   time=27.988..31.772 rows=6.00 loops=1)
   Filter: (ranked_stations.rn = 1)
   Buffers: shared hit=1662
   ->  WindowAgg  (cost=4682.66..4777.73 rows=360 width=47) (actual time=27.986..31.770
     rows=6.00 loops=1)
     Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY
       collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED
       PRECEDING)
     Run Condition: (row_number() OVER w1 <= 1)
     Storage: Memory  Maximum Storage: 17kB
     Buffers: shared hit=1662
     ->  Incremental Sort  (cost=4682.39..4770.53 rows=360 width=39) (actual
       time=27.975..31.746 rows=279.00 loops=1)
       Sort Key: collection_counts.body_type, collection_counts.collections DESC,
         collection_counts.station_id
       Presorted Key: collection_counts.body_type
       Full-sort Groups: 6  Sort Method: quicksort  Average Memory: 28kB  Peak Memory:
         28kB
       Buffers: shared hit=1662
       ->  Subquery Scan on collection_counts  (cost=4681.99..4759.93 rows=360 width=39)
         (actual time=27.937..31.667 rows=279.00 loops=1)
         Buffers: shared hit=1662
         ->  Finalize GroupAggregate  (cost=4681.99..4759.93 rows=360 width=39) (actual
           time=27.935..31.652 rows=279.00 loops=1)
           Group Key: m.body_type, s.station_id
           Buffers: shared hit=1662
           ->  Gather Merge  (cost=4681.99..4751.74 rows=612 width=39) (actual
             time=27.930..31.597 rows=558.00 loops=1)
             Workers Planned: 1
             Workers Launched: 1
             Buffers: shared hit=1662
```

Source: 1.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.1(2)	Marks: /0.5

PostgreSQL execution transcript 1.1.2_2 | part 2/3

```
->  Sort  (cost=3681.98..3682.88 rows=360 width=39) (actual
  time=18.952..18.961 rows=279.00 loops=2)
  Sort Key: m.body_type, s.station_id
  Sort Method: quicksort  Memory: 41kB
  Buffers: shared hit=1662
  Worker 0:  Sort Method: quicksort  Memory: 41kB
  ->  Partial HashAggregate  (cost=3663.10..3666.70 rows=360 width=39)
    (actual time=18.822..18.843 rows=279.00 loops=2)
    Group Key: m.body_type, s.station_id
    Batches: 1  Memory Usage: 72kB
    Buffers: shared hit=1654
    Worker 0:  Batches: 1  Memory Usage: 72kB
    ->  Hash Join  (cost=32.62..3400.27 rows=35044 width=31) (actual
      time=0.697..15.367 rows=29769.50 loops=2)
      Hash Cond: ((v.station_id)::text = (s.station_id)::text)
      Buffers: shared hit=1654
      ->  Hash Join  (cost=30.27..3299.43 rows=35044 width=14) (actual
        time=0.550..12.549 rows=29769.50 loops=2)
        Hash Cond: ((v.model_code)::text = (m.model_code)::text)
        Buffers: shared hit=1652
        ->  Hash Join  (cost=27.25..3200.09 rows=35044 width=15) (actual
          time=0.368..9.712 rows=29769.50 loops=2)
          Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
          Buffers: shared hit=1650
          ->  Parallel Seq Scan on booking_events e  (cost=0.00..3080.41
            rows=35044 width=9) (actual time=0.008..6.405 rows=29769.50
            loops=2)
            Filter: ((event_type)::text = 'Collect'::text)
            Rows Removed by Filter: 68450
            Buffers: shared hit=1636
          ->  Hash  (cost=16.00..16.00 rows=900 width=24) (actual
            time=0.354..0.354 rows=900.00 loops=2)
            Buckets: 1024  Batches: 1  Memory Usage: 58kB
            Buffers: shared hit=14
            ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=24)
             (actual time=0.229..0.280 rows=900.00 loops=2)
              Buffers: shared hit=14
```

Source: 1.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.1(2)	Marks: /0.5

PostgreSQL execution transcript 1.1.2_2 | part 3/3

```
                      ->  Hash  (cost=1.90..1.90 rows=90 width=15) (actual
                        time=0.177..0.178 rows=90.00 loops=2)
                        Buckets: 1024  Batches: 1  Memory Usage: 13kB
                        Buffers: shared hit=2
                        ->  Seq Scan on models m  (cost=0.00..1.90 rows=90 width=15)
                          (actual time=0.164..0.168 rows=90.00 loops=2)
                          Buffers: shared hit=2
                    ->  Hash  (cost=1.60..1.60 rows=60 width=24) (actual time=0.143..0.144
                      rows=60.00 loops=2)
                      Buckets: 1024  Batches: 1  Memory Usage: 12kB
                      Buffers: shared hit=2
                      ->  Seq Scan on stations s  (cost=0.00..1.60 rows=60 width=24)
                        (actual time=0.130..0.133 rows=60.00 loops=2)
                        Buffers: shared hit=2
Planning:
 Buffers: shared hit=335
Planning Time: 3.490 ms
Execution Time: 31.959 ms
(68 rows)
```

Source: 1.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The view returns six rows. Sedan at Carindale Park leads with 973 collections, followed by SUV 838, Hatchback 794, People Mover 510, Van 384 and Ute 333. The top node estimates 2 rows but returns 6; its cost is 4782.24..4782.25 and execution takes 31.959 ms.

A Parallel Seq Scan filters BOOKING_EVENTS to Collect events. Its 29,769.5 actual rows per loop across two loops represent 59,539 collections in total. Hash joins attach vehicle, model and station attributes. Partial aggregation and Finalize GroupAggregate reduce these events to 279 body-type/station groups; incremental sorting and WindowAgg select one winner per body type, and the final sort orders the six winners. The small final result does not avoid processing the underlying event log.

Screenshot Q1.2	Marks: /1.5

PostgreSQL execution transcript 1.2_1 | part 1/1

```
CREATE VIEW V_ACCOUNT_LIABILITY AS
WITH RECURSIVE account_tree AS (
    SELECT member_id, member_id AS account_id
    FROM MEMBERS
    WHERE sponsor_id IS NULL
    UNION ALL
    SELECT child.member_id, parent.account_id
    FROM MEMBERS child
    JOIN account_tree parent ON child.sponsor_id = parent.member_id
)
SELECT root.member_id, root.member_name,
       SUM(e.amount) AS total_amount
FROM account_tree t
JOIN MEMBERS root ON root.member_id = t.account_id
JOIN BOOKING_EVENTS e ON e.member_id = t.member_id
WHERE e.event_type IN ('Return', 'Incident')
  AND e.event_time >= TIMESTAMP '2026-01-01'
  AND e.event_time <  TIMESTAMP '2026-07-01'
GROUP BY root.member_id, root.member_name
ORDER BY total_amount DESC, root.member_id
LIMIT 10;
CREATE VIEW
```

Source: 1.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The recursive seed maps each unsponsored account to itself. Each recursive step maps a sponsored child to the same root account as its parent, so every member in the supplied sponsorship forest maps to exactly one ultimately liable account. Joining charges to this mapping bills personal charges and every level of sponsored charges to the root, without counting an event at several intermediate accounts.

Only Return fares and Incident damage charges are included. The half-open timestamp range from 2026-01-01 up to, but excluding, 2026-07-01 includes all of 30 June. Totals are sorted descending, with member_id as a deterministic tie-breaker, and limited to ten accounts. The result is shown in Q1.3 Query A. An independent upward traversal produced the same ten rows.

Screenshot Q1.3 (2)	Marks: /0.5

Q1.3 part 1 creates MV_ACCOUNT_LIABILITY with the same recursive query. Query A and Query B and their full plans follow.

PostgreSQL execution transcript 1.3.1_1 | part 1/1

```
CREATE MATERIALIZED VIEW MV_ACCOUNT_LIABILITY AS
WITH RECURSIVE account_tree AS (
    SELECT member_id, member_id AS account_id
    FROM MEMBERS
    WHERE sponsor_id IS NULL
    UNION ALL
    SELECT child.member_id, parent.account_id
    FROM MEMBERS child
    JOIN account_tree parent ON child.sponsor_id = parent.member_id
)
SELECT root.member_id, root.member_name,
       SUM(e.amount) AS total_amount
FROM account_tree t
JOIN MEMBERS root ON root.member_id = t.account_id
JOIN BOOKING_EVENTS e ON e.member_id = t.member_id
WHERE e.event_type IN ('Return', 'Incident')
  AND e.event_time >= TIMESTAMP '2026-01-01'
  AND e.event_time <  TIMESTAMP '2026-07-01'
GROUP BY root.member_id, root.member_name
ORDER BY total_amount DESC, root.member_id
LIMIT 10;
SELECT 10
```

Source: 1.3.1_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.3.2_1 | part 1/1

```
-- Query A
SELECT * FROM V_ACCOUNT_LIABILITY;
 member_id |  member_name   | total_amount
-----------+----------------+--------------
     14881 | Ana Lopez      |      6214.77
     13572 | Samuel Choi    |      5629.17
       414 | Nikau Anderson |      4903.24
     12727 | Rohan Wilson   |      4745.38
      6147 | Lily Kim       |      4459.37
      6558 | Tomas Ali      |      4387.82
      5543 | Aisha Robinson |      4025.80
     10682 | Rohan Novak    |      3965.63
     11445 | Minh Bianchi   |      3864.68
       397 | Nikau White    |      3834.32
(10 rows)
```

Source: 1.3.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.3 (2)	Marks: /0.5

PostgreSQL execution transcript 1.3.2_2 | part 1/2

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM V_ACCOUNT_LIABILITY;
QUERY PLAN
------------------------------------------------------------------------------------------
Limit  (cost=389551.20..389551.22 rows=10 width=48) (actual time=47.796..47.802 rows=10.00
  loops=1)
 Buffers: shared hit=2476
 CTE account_tree
 ->  Recursive Union  (cost=0.00..102462.80 rows=1868540 width=8) (actual
   time=0.019..7.426 rows=24000.00 loops=1)
   Storage: Memory  Maximum Storage: 860kB
   Buffers: shared hit=556
   ->  Seq Scan on members  (cost=0.00..518.00 rows=15600 width=8) (actual
     time=0.017..1.335 rows=15600.00 loops=1)
     Filter: (sponsor_id IS NULL)
     Rows Removed by Filter: 8400
     Buffers: shared hit=278
   ->  Hash Join  (cost=818.00..8325.94 rows=185294 width=8) (actual time=0.542..1.166
     rows=2100.00 loops=4)
     Hash Cond: (parent.member_id = child.sponsor_id)
     Buffers: shared hit=278
     ->  WorkTable Scan on account_tree parent  (cost=0.00..3120.00 rows=156000 width=8)
       (actual time=0.001..0.143 rows=6000.00 loops=4)
     ->  Hash  (cost=518.00..518.00 rows=24000 width=8) (actual time=2.055..2.055
       rows=8400.00 loops=1)
       Buckets: 32768  Batches: 1  Memory Usage: 585kB
       Buffers: shared hit=278
       ->  Seq Scan on members child  (cost=0.00..518.00 rows=24000 width=8) (actual
         time=0.023..1.136 rows=24000.00 loops=1)
         Buffers: shared hit=278
 ->  Sort  (cost=287088.40..287148.40 rows=24000 width=48) (actual time=47.794..47.797
   rows=10.00 loops=1)
   Sort Key: (sum(e.amount)) DESC, root.member_id
   Sort Method: top-N heapsort  Memory: 26kB
   Buffers: shared hit=2476
   ->  HashAggregate  (cost=286269.77..286569.77 rows=24000 width=48) (actual
     time=44.514..47.021 rows=10175.00 loops=1)
     Group Key: root.member_id
     Batches: 1  Memory Usage: 4625kB
     Buffers: shared hit=2470
```

Source: 1.3.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.3 (2)	Marks: /0.5

PostgreSQL execution transcript 1.3.2_2 | part 2/2

```
     ->  Hash Join  (cost=6274.50..270093.43 rows=3235267 width=22) (actual
       time=20.048..38.634 rows=30275.00 loops=1)
       Hash Cond: (t.member_id = e.member_id)
       Buffers: shared hit=2470
       ->  Hash Join  (cost=818.00..43094.59 rows=1868540 width=20) (actual
         time=3.500..16.848 rows=24000.00 loops=1)
         Hash Cond: (t.account_id = root.member_id)
         Buffers: shared hit=834
         ->  CTE Scan on account_tree t  (cost=0.00..37370.80 rows=1868540 width=8)
           (actual time=0.021..9.653 rows=24000.00 loops=1)
           Storage: Memory  Maximum Storage: 1007kB
           Buffers: shared hit=556
         ->  Hash  (cost=518.00..518.00 rows=24000 width=16) (actual time=3.436..3.436
           rows=24000.00 loops=1)
           Buckets: 32768  Batches: 1  Memory Usage: 1400kB
           Buffers: shared hit=278
           ->  Seq Scan on members root  (cost=0.00..518.00 rows=24000 width=16) (actual
             time=0.003..1.708 rows=24000.00 loops=1)
             Buffers: shared hit=278
       ->  Hash  (cost=5073.70..5073.70 rows=30624 width=10) (actual time=16.502..16.504
         rows=30275.00 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 1557kB
         Buffers: shared hit=1636
         ->  Seq Scan on booking_events e  (cost=0.00..5073.70 rows=30624 width=10)
           (actual time=7.001..14.264 rows=30275.00 loops=1)
           Filter: (((event_type)::text = ANY ('{Return,Incident}'::text[])) AND
             (event_time >= '2026-01-01 00:00:00'::timestamp without time zone) AND
             (event_time < '2026-07-01 00:00:00'::timestamp without time zone))
           Rows Removed by Filter: 166165
           Buffers: shared hit=1636
Planning:
 Buffers: shared hit=374
Planning Time: 2.616 ms
Execution Time: 48.775 ms
(52 rows)
```

Source: 1.3.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.3.2_3 | part 1/1

```
-- Query B
SELECT * FROM MV_ACCOUNT_LIABILITY;
 member_id |  member_name   | total_amount
-----------+----------------+--------------
     14881 | Ana Lopez      |      6214.77
     13572 | Samuel Choi    |      5629.17
       414 | Nikau Anderson |      4903.24
     12727 | Rohan Wilson   |      4745.38
      6147 | Lily Kim       |      4459.37
      6558 | Tomas Ali      |      4387.82
      5543 | Aisha Robinson |      4025.80
     10682 | Rohan Novak    |      3965.63
     11445 | Minh Bianchi   |      3864.68
       397 | Nikau White    |      3834.32
(10 rows)
```

Source: 1.3.2_3.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.3 (2)	Marks: /0.5

PostgreSQL execution transcript 1.3.2_4 | part 1/1

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM MV_ACCOUNT_LIABILITY;
QUERY PLAN
-------------------------------------------------------------------------------------------
Seq Scan on mv_account_liability  (cost=0.00..18.50 rows=850 width=68) (actual
  time=0.008..0.009 rows=10.00 loops=1)
 Buffers: shared hit=1
Planning:
 Buffers: shared hit=35
Planning Time: 0.580 ms
Execution Time: 0.055 ms
(6 rows)
```

Source: 1.3.2_4.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Both queries initially return the same ten accounts, led by Ana Lopez (14881) with 6,214.77. Query A expands the ordinary view into Recursive Union, joins, HashAggregate and a top-N sort. It builds the 24,000-member account mapping and aggregates 30,275 eligible events. Its top cost is 389551.20..389551.22; execution takes 48.775 ms.

Query B scans only the stored ten-row materialized result, with cost 0.00..18.50 and execution time 0.055 ms. The plan estimates 850 rows but actually returns 10 because the newly created materialized relation has not been analyzed. This estimate does not mean that 850 result rows were stored. The materialized view avoids recomputing recursion, joins and aggregation at read time. SQL consumers needing guaranteed output order should use an explicit ORDER BY when reading a stored relation.

Screenshot Q1.3 (3)	Marks: /0.5

PostgreSQL execution transcript 1.3.3_1 | part 1/1

```
INSERT INTO BOOKING_EVENTS (member_id, vehicle_id, event_type, event_time, amount)
SELECT member_id, 'VEH-0001', 'Incident', TIMESTAMP '2026-05-04 09:15:00', 90000.00
FROM MEMBERS WHERE sponsor_id IS NOT NULL ORDER BY member_id DESC LIMIT 1;
INSERT 0 1
```

Source: 1.3.3_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.3.3_2 | part 1/1

```
-- Query A reflects the inserted charge immediately.
SELECT * FROM V_ACCOUNT_LIABILITY;
 member_id |  member_name   | total_amount
-----------+----------------+--------------
     13943 | Tuan Haddad    |     90673.75
     14881 | Ana Lopez      |      6214.77
     13572 | Samuel Choi    |      5629.17
       414 | Nikau Anderson |      4903.24
     12727 | Rohan Wilson   |      4745.38
      6147 | Lily Kim       |      4459.37
      6558 | Tomas Ali      |      4387.82
      5543 | Aisha Robinson |      4025.80
     10682 | Rohan Novak    |      3965.63
     11445 | Minh Bianchi   |      3864.68
(10 rows)
```

Source: 1.3.3_2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.3.3_3 | part 1/1

```
-- Query B still contains its stored result.
SELECT * FROM MV_ACCOUNT_LIABILITY;
 member_id |  member_name   | total_amount
-----------+----------------+--------------
     14881 | Ana Lopez      |      6214.77
     13572 | Samuel Choi    |      5629.17
       414 | Nikau Anderson |      4903.24
     12727 | Rohan Wilson   |      4745.38
      6147 | Lily Kim       |      4459.37
      6558 | Tomas Ali      |      4387.82
      5543 | Aisha Robinson |      4025.80
     10682 | Rohan Novak    |      3965.63
     11445 | Minh Bianchi   |      3864.68
       397 | Nikau White    |      3834.32
(10 rows)
```

Source: 1.3.3_3.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 1.3.3_4 | part 1/1

```
REFRESH MATERIALIZED VIEW MV_ACCOUNT_LIABILITY;
REFRESH MATERIALIZED VIEW
```

Source: 1.3.3_4.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q1.3 (3)	Marks: /0.5

PostgreSQL execution transcript 1.3.3_5 | part 1/1

```
-- Query B after refresh.
SELECT * FROM MV_ACCOUNT_LIABILITY;
 member_id |  member_name   | total_amount
-----------+----------------+--------------
     13943 | Tuan Haddad    |     90673.75
     14881 | Ana Lopez      |      6214.77
     13572 | Samuel Choi    |      5629.17
       414 | Nikau Anderson |      4903.24
     12727 | Rohan Wilson   |      4745.38
      6147 | Lily Kim       |      4459.37
      6558 | Tomas Ali      |      4387.82
      5543 | Aisha Robinson |      4025.80
     10682 | Rohan Novak    |      3965.63
     11445 | Minh Bianchi   |      3864.68
(10 rows)
```

Source: 1.3.3_5.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The supplied INSERT chooses member 24000. Its sponsorship chain is 24000 -> 22086 -> 18148 -> 13943, so the additional 90,000.00 is billed to Tuan Haddad (13943). Query A immediately puts that account first at 90,673.75. Query B still shows the earlier top ten, led by Ana Lopez, and does not include Tuan Haddad. After REFRESH, Query B shows the same updated ten rows as Query A; account 397 drops out of the top ten.

A materialized view trades faster reads for extra storage, refresh work and potentially stale answers between refreshes. An ordinary view computes an answer from the data visible to each query.

Screenshot Q2.1 (1)	Marks: /0.5

PostgreSQL execution transcript 2.1.1_1 | part 1/1

```
CREATE INDEX IDX_EVENT_TYPE_TIME
ON BOOKING_EVENTS (event_type, event_time);
CREATE INDEX
```

Source: 2.1.1_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

This B-tree index has event_type as its first key and event_time as its second key, in the required order. It orders timestamps within each event type.

Screenshot Q2.1 (2)	Marks: /1.5

PostgreSQL execution transcript 2.1.2_1 | part 1/1

```
SELECT * FROM V_BUSIEST_BAYS;
  body_type   |  station_name  | collections
--------------+----------------+-------------
 Sedan        | Carindale Park |         973
 SUV          | Chermside Park |         838
 Hatchback    | Chermside Park |         794
 People Mover | St Lucia Hub   |         510
 Van          | Chermside Park |         384
 Ute          | Carindale Park |         333
(6 rows)
```

Source: 2.1.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 2.1.2_2 | part 1/3

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM V_BUSIEST_BAYS;
QUERY PLAN
-----------------------------------------------------------------------------------------------
Sort  (cost=4782.25..4782.25 rows=2 width=32) (actual time=27.529..31.381 rows=6.00
  loops=1)
 Sort Key: ranked_stations.collections DESC, ranked_stations.body_type
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1665
 ->  Subquery Scan on ranked_stations  (cost=4682.66..4782.24 rows=2 width=32) (actual
   time=27.345..31.369 rows=6.00 loops=1)
   Filter: (ranked_stations.rn = 1)
   Buffers: shared hit=1662
   ->  WindowAgg  (cost=4682.66..4777.74 rows=360 width=47) (actual time=27.344..31.367
     rows=6.00 loops=1)
     Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY
       collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED
       PRECEDING)
     Run Condition: (row_number() OVER w1 <= 1)
     Storage: Memory  Maximum Storage: 17kB
     Buffers: shared hit=1662
     ->  Incremental Sort  (cost=4682.40..4770.54 rows=360 width=39) (actual
       time=27.334..31.343 rows=279.00 loops=1)
       Sort Key: collection_counts.body_type, collection_counts.collections DESC,
         collection_counts.station_id
       Presorted Key: collection_counts.body_type
       Full-sort Groups: 6  Sort Method: quicksort  Average Memory: 28kB  Peak Memory:
         28kB
       Buffers: shared hit=1662
       ->  Subquery Scan on collection_counts  (cost=4682.00..4759.94 rows=360 width=39)
         (actual time=27.295..31.265 rows=279.00 loops=1)
         Buffers: shared hit=1662
         ->  Finalize GroupAggregate  (cost=4682.00..4759.94 rows=360 width=39) (actual
           time=27.294..31.243 rows=279.00 loops=1)
           Group Key: m.body_type, s.station_id
           Buffers: shared hit=1662
           ->  Gather Merge  (cost=4682.00..4751.75 rows=612 width=39) (actual
             time=27.288..31.190 rows=558.00 loops=1)
             Workers Planned: 1
             Workers Launched: 1
             Buffers: shared hit=1662
```

Source: 2.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q2.1 (2)	Marks: /1.5

PostgreSQL execution transcript 2.1.2_2 | part 2/3

```
->  Sort  (cost=3681.99..3682.89 rows=360 width=39) (actual
  time=17.672..17.681 rows=279.00 loops=2)
  Sort Key: m.body_type, s.station_id
  Sort Method: quicksort  Memory: 41kB
  Buffers: shared hit=1662
  Worker 0:  Sort Method: quicksort  Memory: 41kB
  ->  Partial HashAggregate  (cost=3663.10..3666.70 rows=360 width=39)
    (actual time=17.547..17.565 rows=279.00 loops=2)
    Group Key: m.body_type, s.station_id
    Batches: 1  Memory Usage: 72kB
    Buffers: shared hit=1654
    Worker 0:  Batches: 1  Memory Usage: 72kB
    ->  Hash Join  (cost=32.62..3400.27 rows=35044 width=31) (actual
      time=0.622..14.352 rows=29769.50 loops=2)
      Hash Cond: ((v.station_id)::text = (s.station_id)::text)
      Buffers: shared hit=1654
      ->  Hash Join  (cost=30.27..3299.44 rows=35044 width=14) (actual
        time=0.410..11.564 rows=29769.50 loops=2)
        Hash Cond: ((v.model_code)::text = (m.model_code)::text)
        Buffers: shared hit=1652
        ->  Hash Join  (cost=27.25..3200.09 rows=35044 width=15) (actual
          time=0.253..8.882 rows=29769.50 loops=2)
          Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
          Buffers: shared hit=1650
          ->  Parallel Seq Scan on booking_events e  (cost=0.00..3080.42
            rows=35044 width=9) (actual time=0.008..5.886 rows=29769.50
            loops=2)
            Filter: ((event_type)::text = 'Collect'::text)
            Rows Removed by Filter: 68451
            Buffers: shared hit=1636
          ->  Hash  (cost=16.00..16.00 rows=900 width=24) (actual
            time=0.238..0.238 rows=900.00 loops=2)
            Buckets: 1024  Batches: 1  Memory Usage: 58kB
            Buffers: shared hit=14
            ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=24)
             (actual time=0.119..0.170 rows=900.00 loops=2)
              Buffers: shared hit=14
```

Source: 2.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q2.1 (2)	Marks: /1.5

PostgreSQL execution transcript 2.1.2_2 | part 3/3

```
                      ->  Hash  (cost=1.90..1.90 rows=90 width=15) (actual
                        time=0.152..0.152 rows=90.00 loops=2)
                        Buckets: 1024  Batches: 1  Memory Usage: 13kB
                        Buffers: shared hit=2
                        ->  Seq Scan on models m  (cost=0.00..1.90 rows=90 width=15)
                          (actual time=0.139..0.143 rows=90.00 loops=2)
                          Buffers: shared hit=2
                    ->  Hash  (cost=1.60..1.60 rows=60 width=24) (actual time=0.207..0.207
                      rows=60.00 loops=2)
                      Buckets: 1024  Batches: 1  Memory Usage: 12kB
                      Buffers: shared hit=2
                      ->  Seq Scan on stations s  (cost=0.00..1.60 rows=60 width=24)
                        (actual time=0.191..0.194 rows=60.00 loops=2)
                        Buffers: shared hit=2
Planning:
 Buffers: shared hit=367
Planning Time: 4.397 ms
Execution Time: 31.582 ms
(68 rows)
```

Source: 2.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The six result rows are unchanged. The plan remains a Parallel Seq Scan of the event log followed by hash joins, aggregation, ranking and sorting; IDX_EVENT_TYPE_TIME is not used. Its top cost is 4782.25..4782.25 versus 4782.24..4782.25 before the index, and execution is 31.582 ms versus 31.959 ms. These small timing differences are single-run variation, not evidence of an index benefit.

Collect matches 59,539 of the 196,441 events now present, about 30.31 percent. Those events are distributed throughout the heap, and the query also needs vehicle_id, which this index does not contain. An index path would traverse many index entries and fetch many heap pages, whereas a parallel sequential scan reads the table efficiently. There is no timestamp condition, so the second index column does not narrow this query. Adding an index gives the planner another option; it does not require its use.

Screenshot Q2.1 (3)	Marks: /1

PostgreSQL execution transcript 2.1.3_1 | part 1/1

```
SELECT count(*), sum(amount) FROM BOOKING_EVENTS
WHERE event_type = 'Incident'
  AND event_time >= TIMESTAMP '2026-03-01'
  AND event_time <  TIMESTAMP '2026-04-01';
 count |   sum
-------+----------
   116 | 86382.67
(1 row)
```

Source: 2.1.3_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 2.1.3_2 | part 1/1

```
EXPLAIN (ANALYZE, BUFFERS) SELECT count(*), sum(amount) FROM BOOKING_EVENTS
WHERE event_type = 'Incident'
AND event_time >= TIMESTAMP '2026-03-01'
AND event_time <  TIMESTAMP '2026-04-01';
QUERY PLAN
------------------------------------------------------------------------------------------
Aggregate  (cost=410.92..410.93 rows=1 width=40) (actual time=0.244..0.244 rows=1.00
  loops=1)
 Buffers: shared hit=82
 ->  Bitmap Heap Scan on booking_events  (cost=6.10..410.26 rows=132 width=6) (actual
   time=0.051..0.207 rows=116.00 loops=1)
   Recheck Cond: (((event_type)::text = 'Incident'::text) AND (event_time >= '2026-03-01
     00:00:00'::timestamp without time zone) AND (event_time < '2026-04-01
     00:00:00'::timestamp without time zone))
   Heap Blocks: exact=79
   Buffers: shared hit=82
   ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..6.07 rows=132 width=0)
     (actual time=0.034..0.034 rows=116.00 loops=1)
     Index Cond: (((event_type)::text = 'Incident'::text) AND (event_time >= '2026-03-01
       00:00:00'::timestamp without time zone) AND (event_time < '2026-04-01
       00:00:00'::timestamp without time zone))
     Index Searches: 1
     Buffers: shared hit=3
Planning:
 Buffers: shared hit=126
Planning Time: 1.412 ms
Execution Time: 0.292 ms
(14 rows)
```

Source: 2.1.3_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The query returns count 116 and sum 86,382.67. The aggregate estimates and returns one result row, with cost 410.92..410.93 and execution time 0.292 ms. Its child Bitmap Index Scan finds 116 matching entries, versus 132 estimated, and Bitmap Heap Scan visits 79 exact heap blocks before computing the aggregate.

Incident equality fixes the leading key, and the lower and upper March timestamp bounds restrict the second key to a short contiguous range within Incident entries. These 116 events are about 0.059 percent of the table, so the bitmap index path avoids most rows and pages. The plan needs heap access for amount, which is not stored in the index. This is much more selective than the all-time Collect query in Q2.1 part 2.

Screenshot Q2.2(1)	Marks: /1

PostgreSQL execution transcript 2.2.1_1 | part 1/1

```
SELECT LOWER(SPLIT_PART(email, '@', 2)) AS provider_domain,
       COUNT(*) AS member_count
FROM MEMBERS
GROUP BY LOWER(SPLIT_PART(email, '@', 2))
ORDER BY member_count DESC, provider_domain;
 provider_domain  | member_count
------------------+--------------
 gmail.com        |         8164
 outlook.com      |         3388
 hotmail.com      |         2200
 yahoo.com.au     |         1937
 bigpond.com      |         1686
 icloud.com       |         1652
 uqconnect.edu.au |         1486
 protonmail.com   |         1165
 optusnet.com.au  |          901
 iinet.net.au     |          733
 tpg.com.au       |          440
 exemail.com.au   |          248
(12 rows)
```

Source: 2.2.1_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

LOWER(SPLIT_PART(email, '@', 2)) extracts the part after @ and normalizes its case before grouping. The query reports 12 domains accounting for all 24,000 members, with gmail.com first at 8,164 members. exemail.com.au has 248 members. The secondary alphabetical ordering makes equal counts reproducible.

Screenshot Q2.2 (2)	Marks: /1

PostgreSQL execution transcript 2.2.2_1 | part 1/1

```
EXPLAIN (ANALYZE, BUFFERS) SELECT LOWER(SPLIT_PART(email, '@', 2)) AS provider_domain,
  COUNT(*) AS member_count
FROM MEMBERS
GROUP BY LOWER(SPLIT_PART(email, '@', 2))
ORDER BY member_count DESC, provider_domain;
QUERY PLAN
----------------------------------------------------------------------------------------
Sort  (cost=2864.09..2924.09 rows=24000 width=40) (actual time=4.576..4.577 rows=12.00
  loops=1)
 Sort Key: (count(*)) DESC, (lower(split_part((email)::text, '@'::text, 2)))
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=284
 ->  HashAggregate  (cost=758.00..1118.00 rows=24000 width=40) (actual time=4.530..4.554
   rows=12.00 loops=1)
   Group Key: lower(split_part((email)::text, '@'::text, 2))
   Batches: 1  Memory Usage: 545kB
   Buffers: shared hit=278
   ->  Seq Scan on members  (cost=0.00..638.00 rows=24000 width=32) (actual
     time=0.020..3.077 rows=24000.00 loops=1)
     Buffers: shared hit=278
Planning:
 Buffers: shared hit=139
Planning Time: 1.238 ms
Execution Time: 4.736 ms
(14 rows)
```

Source: 2.2.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The plan performs a Seq Scan of all 24,000 MEMBERS rows, computes the normalized domain, uses HashAggregate for the counts, and sorts the resulting 12 groups by count descending. The top cost is 2864.09..2924.09 and execution time is 4.736 ms. It estimates 24,000 groups but actually returns only 12, reflecting the lack of statistics for this derived expression at this point.

Every member contributes to the requested whole-population counts, so a complete scan is appropriate. Hash aggregation has a single batch and the final quicksort uses only 25 kB; neither step spills to disk in this run.

Screenshot Q2.2 (3)	Marks: /1

The UNIQUE B-tree on email orders complete addresses from their first character, so members sharing a domain can be scattered across many different address prefixes. It cannot provide a selective lookup or grouping order on the normalized domain expression; although a full index-only scan could still read every email and compute the domain, it would still have to examine the whole population.

Comments

No SQL file is required for this explanation-only part.

Screenshot Q2.2 (4)	Marks: /0.5

PostgreSQL execution transcript 2.2.4_1 | part 1/1

```
CREATE INDEX IDX_MEMBER_DOMAIN
ON MEMBERS (LOWER(SPLIT_PART(email, '@', 2)));
CREATE INDEX
```

Source: 2.2.4_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The expression in IDX_MEMBER_DOMAIN exactly matches LOWER(SPLIT_PART(email, '@', 2)) in the queries, including the case folding. PostgreSQL can therefore look up a normalized domain directly in this index.

Screenshot Q2.2 (5)	Marks: /1.5

The domain-count query keeps its Seq Scan -> HashAggregate -> Sort plan, estimates 24,000 groups and returns 12. Its top cost stays 2864.09..2924.09; execution is 4.825 ms compared with 4.736 ms before the index. All 24,000 members still have to contribute to the counts, and an index ordered by domain cannot directly satisfy ORDER BY COUNT(*) DESC.

The rare-provider lookup instead uses Bitmap Index Scan on IDX_MEMBER_DOMAIN followed by Bitmap Heap Scan. It returns 248 members, about 1.03 percent of MEMBERS, versus 120 estimated, with cost 5.22..226.08 and execution time 0.440 ms. The bitmap batches access to 164 exact heap blocks; heap access is needed for member_id, member_name and email. The index avoids evaluating the domain predicate on every member. Its advantage is selective access, rather than speeding every query that mentions the expression.

PostgreSQL execution transcript 2.2.5_1 | part 1/1

```
EXPLAIN (ANALYZE, BUFFERS) SELECT LOWER(SPLIT_PART(email, '@', 2)) AS provider_domain,
  COUNT(*) AS member_count
FROM MEMBERS
GROUP BY LOWER(SPLIT_PART(email, '@', 2))
ORDER BY member_count DESC, provider_domain;
QUERY PLAN
------------------------------------------------------------------------------------------
Sort  (cost=2864.09..2924.09 rows=24000 width=40) (actual time=4.664..4.665 rows=12.00
  loops=1)
 Sort Key: (count(*)) DESC, (lower(split_part((email)::text, '@'::text, 2)))
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=284
 ->  HashAggregate  (cost=758.00..1118.00 rows=24000 width=40) (actual time=4.619..4.641
   rows=12.00 loops=1)
   Group Key: lower(split_part((email)::text, '@'::text, 2))
   Batches: 1  Memory Usage: 545kB
   Buffers: shared hit=278
   ->  Seq Scan on members  (cost=0.00..638.00 rows=24000 width=32) (actual
     time=0.020..3.136 rows=24000.00 loops=1)
     Buffers: shared hit=278
Planning:
 Buffers: shared hit=153 read=1
Planning Time: 2.017 ms
Execution Time: 4.825 ms
(14 rows)
```

Source: 2.2.5_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q2.2 (5)	Marks: /1.5

PostgreSQL execution transcript 2.2.5_2 | part 1/1

```
SELECT member_id, member_name, email FROM MEMBERS
WHERE LOWER(SPLIT_PART(email, '@', 2)) = 'exemail.com.au';
 member_id |     member_name     |                email
-----------+---------------------+--------------------------------------
         3 | Elena O'Brien       | elena.obrien3@exemail.com.au
        21 | Minh Sullivan       | minh.sullivan21@exemail.com.au
       123 | Amelia Kelly        | amelia.kelly123@exemail.com.au
       479 | Kiri Liu            | kiri.liu479@exemail.com.au
       507 | Cillian Dubois      | Cillian.dubois507@Exemail.com.au
       526 | Marcus Novak        | marcus.novak526@exemail.com.au
       622 | Sanjay Ivanova      | sanjay.ivanova622@exemail.com.au
       678 | Lucas Larsen        | lucas.larsen678@exemail.com.au
       806 | Thomas Walker       | thomas.walker806@exemail.com.au
       833 | Minh Chen           | Minh.chen833@Exemail.com.au
... 228 intermediate rows omitted; first 10 and last 10 shown ...
     23294 | Hiro Wilson         | hiro.wilson23294@exemail.com.au
     23409 | Willow Okafor       | willow.okafor23409@exemail.com.au
     23507 | Yusuf Kowalski      | yusuf.kowalski23507@exemail.com.au
     23598 | Ethan Garcia        | Ethan.garcia23598@Exemail.com.au
     23655 | Amara Mokoena       | amara.mokoena23655@exemail.com.au
     23797 | Linh Zhang          | linh.zhang23797@exemail.com.au
     23806 | Ava Johnson         | Ava.johnson23806@Exemail.com.au
     23816 | Elena Whetu         | elena.whetu23816@exemail.com.au
     23921 | Yuki Dimitriou      | yuki.dimitriou23921@exemail.com.au
     23930 | Saoirse Hayes       | Saoirse.hayes23930@Exemail.com.au
(248 rows)
```

Source: 2.2.5_2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 2.2.5_3 | part 1/1

```
EXPLAIN (ANALYZE, BUFFERS) SELECT member_id, member_name, email FROM MEMBERS
WHERE LOWER(SPLIT_PART(email, '@', 2)) = 'exemail.com.au';
QUERY PLAN
-------------------------------------------------------------------------------------------
Bitmap Heap Scan on members  (cost=5.22..226.08 rows=120 width=45) (actual
  time=0.044..0.367 rows=248.00 loops=1)
 Recheck Cond: (lower(split_part((email)::text, '@'::text, 2)) = 'exemail.com.au'::text)
 Heap Blocks: exact=164
 Buffers: shared hit=166
 ->  Bitmap Index Scan on idx_member_domain  (cost=0.00..5.19 rows=120 width=0) (actual
   time=0.022..0.022 rows=248.00 loops=1)
   Index Cond: (lower(split_part((email)::text, '@'::text, 2)) = 'exemail.com.au'::text)
   Index Searches: 1
   Buffers: shared hit=2
Planning:
 Buffers: shared hit=125
Planning Time: 1.657 ms
Execution Time: 0.440 ms
(12 rows)
```

Source: 2.2.5_3.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The screenshots above show the full aggregate plan, the first and last ten rare-provider results, and the full selective-lookup plan.

Screenshot Q3.1 (1)	Marks: /1.5

PostgreSQL execution transcript 3.1.1_1 | part 1/1

```
-- Statement A
SELECT v.vehicle_id
FROM VEHICLES v
WHERE NOT EXISTS (
    SELECT 1 FROM BOOKING_EVENTS e
    WHERE e.vehicle_id = v.vehicle_id
      AND e.event_type = 'Collect'
)
ORDER BY v.vehicle_id;
 vehicle_id
------------
 VEH-0019
 VEH-0041
 VEH-0045
 VEH-0049
 VEH-0057
 VEH-0094
 VEH-0098
 VEH-0123
 VEH-0137
 VEH-0139
... 55 intermediate rows omitted; first 10 and last 10 shown ...
 VEH-0820
 VEH-0826
 VEH-0845
 VEH-0857
 VEH-0860
 VEH-0861
 VEH-0874
 VEH-0881
 VEH-0882
 VEH-0891
(75 rows)
```

Source: 3.1.1_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q3.1 (1)	Marks: /1.5

PostgreSQL execution transcript 3.1.1_2 | part 1/1

```
-- Statement B
SELECT v.vehicle_id
FROM VEHICLES v
LEFT JOIN BOOKING_EVENTS e
  ON e.vehicle_id = v.vehicle_id
 AND e.event_type = 'Collect'
WHERE e.vehicle_id IS NULL
ORDER BY v.vehicle_id;
 vehicle_id
------------
 VEH-0019
 VEH-0041
 VEH-0045
 VEH-0049
 VEH-0057
 VEH-0094
 VEH-0098
 VEH-0123
 VEH-0137
 VEH-0139
... 55 intermediate rows omitted; first 10 and last 10 shown ...
 VEH-0820
 VEH-0826
 VEH-0845
 VEH-0857
 VEH-0860
 VEH-0861
 VEH-0874
 VEH-0881
 VEH-0882
 VEH-0891
(75 rows)
```

Source: 3.1.1_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Both statements return the same 75 vehicle IDs, ordered by vehicle_id. Statement A uses a correlated NOT EXISTS test. Statement B places the Collect predicate in the LEFT JOIN condition and keeps rows with no matching event vehicle_id. The matched-side vehicle_id is NOT NULL in the schema, so NULL identifies an unmatched vehicle unambiguously.

Putting event_type in WHERE instead of ON would remove the unmatched rows and break the outer-join solution. Independent EXCEPT ALL checks in both directions found no differences. The screenshots show the first ten and last ten rows of each 75-row result.

Screenshot Q3.1 (2)	Marks: /1.5

PostgreSQL execution transcript 3.1.2_1 | part 1/2

```
-- Statement A
EXPLAIN (ANALYZE, BUFFERS) SELECT v.vehicle_id
FROM VEHICLES v
WHERE NOT EXISTS (
 SELECT 1 FROM BOOKING_EVENTS e
 WHERE e.vehicle_id = v.vehicle_id
  AND e.event_type = 'Collect'
)
ORDER BY v.vehicle_id;
QUERY PLAN
------------------------------------------------------------------------------------------
Sort  (cost=4579.18..4579.30 rows=48 width=9) (actual time=12.704..12.707 rows=75.00
  loops=1)
 Sort Key: v.vehicle_id
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1877
 ->  Hash Right Anti Join  (cost=1413.37..4577.84 rows=48 width=9) (actual
   time=12.659..12.670 rows=75.00 loops=1)
   Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
   Buffers: shared hit=1874
   ->  Bitmap Heap Scan on booking_events e  (cost=1386.12..3766.79 rows=59574 width=9)
     (actual time=1.465..8.881 rows=59539.00 loops=1)
     Recheck Cond: ((event_type)::text = 'Collect'::text)
     Heap Blocks: exact=1636
     Buffers: shared hit=1867
     ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..1371.23 rows=59574 width=0)
       (actual time=1.360..1.360 rows=59539.00 loops=1)
       Index Cond: ((event_type)::text = 'Collect'::text)
       Index Searches: 1
       Buffers: shared hit=231
   ->  Hash  (cost=16.00..16.00 rows=900 width=9) (actual time=0.099..0.100 rows=900.00
     loops=1)
     Buckets: 1024  Batches: 1  Memory Usage: 45kB
     Buffers: shared hit=7
     ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=9) (actual
       time=0.010..0.050 rows=900.00 loops=1)
       Buffers: shared hit=7
Planning:
 Buffers: shared hit=233
```

Source: 3.1.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 3.1.2_1 | part 2/2

```
Planning Time: 2.249 ms
Execution Time: 12.788 ms
(24 rows)
```

Source: 3.1.2_1.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q3.1 (2)	Marks: /1.5

PostgreSQL execution transcript 3.1.2_2 | part 1/1

```
-- Statement B
EXPLAIN (ANALYZE, BUFFERS) SELECT v.vehicle_id
FROM VEHICLES v
LEFT JOIN BOOKING_EVENTS e
ON e.vehicle_id = v.vehicle_id
AND e.event_type = 'Collect'
WHERE e.vehicle_id IS NULL
ORDER BY v.vehicle_id;
QUERY PLAN
------------------------------------------------------------------------------------------
Sort  (cost=4579.18..4579.30 rows=48 width=9) (actual time=12.478..12.482 rows=75.00
  loops=1)
 Sort Key: v.vehicle_id
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1877
 ->  Hash Right Anti Join  (cost=1413.37..4577.84 rows=48 width=9) (actual
   time=12.438..12.448 rows=75.00 loops=1)
   Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
   Buffers: shared hit=1874
   ->  Bitmap Heap Scan on booking_events e  (cost=1386.12..3766.79 rows=59574 width=9)
     (actual time=1.413..8.671 rows=59539.00 loops=1)
     Recheck Cond: ((event_type)::text = 'Collect'::text)
     Heap Blocks: exact=1636
     Buffers: shared hit=1867
     ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..1371.23 rows=59574 width=0)
       (actual time=1.318..1.318 rows=59539.00 loops=1)
       Index Cond: ((event_type)::text = 'Collect'::text)
       Index Searches: 1
       Buffers: shared hit=231
   ->  Hash  (cost=16.00..16.00 rows=900 width=9) (actual time=0.101..0.102 rows=900.00
     loops=1)
     Buckets: 1024  Batches: 1  Memory Usage: 45kB
     Buffers: shared hit=7
     ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=9) (actual
       time=0.010..0.052 rows=900.00 loops=1)
       Buffers: shared hit=7
Planning:
 Buffers: shared hit=233
Planning Time: 2.697 ms
Execution Time: 12.558 ms
(24 rows)
```

Source: 3.1.2_2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

The two plans have the same operator structure and cost 4579.18..4579.30. Both return 75 rows, versus 48 estimated, using Hash Right Anti Join and a final vehicle_id sort. Statement A takes 12.788 ms and Statement B 12.558 ms in these runs.

PostgreSQL decorrelates NOT EXISTS and recognizes the outer-join NULL filter as the same anti-join operation. It hashes the small set of 900 vehicles and scans 59,539 Collect events through a bitmap index/heap path, keeping the vehicle rows without any match. The Right direction describes the physical join inputs chosen by the optimizer; it preserves the intended vehicle-side anti-join semantics. Neither syntax forces a separate full event scan for each vehicle. The minor time difference does not establish a consistent performance difference.

Screenshot Q3.2 (1)	Marks: /2

Each of the three settings controls index, index-only and bitmap scans together. EXPLAIN ANALYZE executes Queries C and D without printing their thousands of tuples. The screenshots include every setting, all six complete plans and the four resets. The comparisons are in Q3.2 part 2.

PostgreSQL execution transcript 3.2 | part 1/6

```
-- Setting 1: index scans enabled; sequential scans enabled.
SET enable_indexscan = on;
SET
SET enable_indexonlyscan = on;
SET
SET enable_bitmapscan = on;
SET
SET enable_seqscan = on;
SET
-- Query C
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type < 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Bitmap Heap Scan on booking_events  (cost=179.44..1910.62 rows=7615 width=38) (actual
  time=0.562..4.431 rows=8158.00 loops=1)
 Recheck Cond: ((event_type)::text < 'Collect'::text)
 Heap Blocks: exact=1625
 Buffers: shared hit=1628 read=31
 ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..177.53 rows=7615 width=0)
   (actual time=0.463..0.464 rows=8158.00 loops=1)
   Index Cond: ((event_type)::text < 'Collect'::text)
   Index Searches: 1
   Buffers: shared hit=3 read=31
Planning:
 Buffers: shared hit=99
Planning Time: 1.580 ms
Execution Time: 4.667 ms
(12 rows)
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 3.2 | part 2/6

```
-- Query D
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type >= 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Seq Scan on booking_events  (cost=0.00..4091.51 rows=188826 width=38) (actual
  time=0.017..10.615 rows=188283.00 loops=1)
 Filter: ((event_type)::text >= 'Collect'::text)
 Rows Removed by Filter: 8158
 Buffers: shared hit=1636
Planning:
 Buffers: shared hit=3
Planning Time: 0.065 ms
Execution Time: 14.145 ms
(8 rows)
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q3.2 (1)	Marks: /2

PostgreSQL execution transcript 3.2 | part 3/6

```
-- Setting 2: index scans enabled; sequential scans suppressed.
SET enable_indexscan = on;
SET
SET enable_indexonlyscan = on;
SET
SET enable_bitmapscan = on;
SET
SET enable_seqscan = off;
SET
-- Query C
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type < 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Bitmap Heap Scan on booking_events  (cost=179.44..1910.62 rows=7615 width=38) (actual
  time=0.403..1.914 rows=8158.00 loops=1)
 Recheck Cond: ((event_type)::text < 'Collect'::text)
 Heap Blocks: exact=1625
 Buffers: shared hit=1659
 ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..177.53 rows=7615 width=0)
   (actual time=0.272..0.272 rows=8158.00 loops=1)
   Index Cond: ((event_type)::text < 'Collect'::text)
   Index Searches: 1
   Buffers: shared hit=34
Planning Time: 0.082 ms
Execution Time: 2.242 ms
(10 rows)
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 3.2 | part 4/6

```
-- Query D
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type >= 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Bitmap Heap Scan on booking_events  (cost=4387.82..8384.15 rows=188826 width=38) (actual
  time=6.280..13.643 rows=188283.00 loops=1)
 Recheck Cond: ((event_type)::text >= 'Collect'::text)
 Heap Blocks: exact=1636
 Buffers: shared hit=1868 read=493
 ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..4340.61 rows=188826 width=0)
   (actual time=6.180..6.181 rows=188283.00 loops=1)
   Index Cond: ((event_type)::text >= 'Collect'::text)
   Index Searches: 1
   Buffers: shared hit=232 read=493
Planning Time: 0.065 ms
Execution Time: 17.270 ms
(10 rows)
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q3.2 (1)	Marks: /2

PostgreSQL execution transcript 3.2 | part 5/6

```
-- Setting 3: index scans suppressed; sequential scans enabled.
SET enable_indexscan = off;
SET
SET enable_indexonlyscan = off;
SET
SET enable_bitmapscan = off;
SET
SET enable_seqscan = on;
SET
-- Query C
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type < 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Seq Scan on booking_events  (cost=0.00..4091.51 rows=7615 width=38) (actual
  time=0.009..7.727 rows=8158.00 loops=1)
 Filter: ((event_type)::text < 'Collect'::text)
 Rows Removed by Filter: 188283
 Buffers: shared hit=1636
Planning Time: 0.066 ms
Execution Time: 7.886 ms
(6 rows)
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript 3.2 | part 6/6

```
-- Query D
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM BOOKING_EVENTS WHERE event_type >= 'Collect';
QUERY PLAN
-------------------------------------------------------------------------------------------
Seq Scan on booking_events  (cost=0.00..4091.51 rows=188826 width=38) (actual
  time=0.008..10.409 rows=188283.00 loops=1)
 Filter: ((event_type)::text >= 'Collect'::text)
 Rows Removed by Filter: 8158
 Buffers: shared hit=1636
Planning Time: 0.073 ms
Execution Time: 13.995 ms
(6 rows)

-- Restore all four scan settings.
SET enable_indexscan = on;
SET
SET enable_indexonlyscan = on;
SET
SET enable_bitmapscan = on;
SET
SET enable_seqscan = on;
SET
```

Source: 3.2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Query C uses Bitmap Heap Scan with a Bitmap Index Scan under settings 1 and 2, and Seq Scan under setting 3. Query D uses Seq Scan under settings 1 and 3, and Bitmap Heap Scan with a Bitmap Index Scan when sequential scans are suppressed in setting 2. The following section explains each combination with its measured cost, time and row count.

Screenshot Q3.2 (2)	Marks: /2

At this stage BOOKING_EVENTS contains 196,441 rows. Under the test database C collation, Query C selects only Cancel: 8,158 rows (4.15 percent). Query D selects Collect, Incident, Reserve and Return: 188,283 rows (95.85 percent). The predicates partition every row because event_type is NOT NULL.

Setting 1, Query C: the bitmap index path is selective and has estimated total cost 1,910.62. It identifies 8,158 event locations and visits 1,625 exact heap blocks, returning 8,158 rows versus 7,615 estimated in 4.667 ms. The many heap blocks show why row selectivity alone does not perfectly predict page savings.

Setting 1, Query D: Seq Scan costs 4,091.51, returns 188,283 rows versus 188,826 estimated, and takes 14.145 ms. Reading nearly the whole heap directly avoids traversing an index and then fetching almost every data page.

Setting 2, Query C: suppressing sequential scans leaves the same bitmap structure and cost 1,910.62; 8,158 rows return in 2.242 ms. The later run hits cached index pages, so the time improvement is not a change of access method.

Setting 2, Query D: the planner now chooses the bitmap index/heap path, with cost 8,384.15 and 188,283 rows in 17.270 ms. It processes 188,283 index entries and all 1,636 heap blocks, incurring extra index work. Its estimated cost exceeds the unrestricted sequential path, and it is slower in this run.

Setting 3, Query C: after all three index scan types are suppressed, Seq Scan costs 4,091.51, returns 8,158 rows and removes 188,283 rows by its filter, taking 7.886 ms. Disabling indexes removes the previously preferred selective access path.

Setting 3, Query D: Seq Scan remains the preferred permitted path, at cost 4,091.51 and 188,283 rows in 13.995 ms. This is the same structure as the unrestricted D plan, with normal timing variation.

The change of inequality changes selectivity, despite using the same table, column and boundary value. The cost model balances index traversal, heap-page access and filtering. SET parameters influence the allowed or discouraged methods; they do not prove one method is best in general. The script restores enable_indexscan, enable_indexonlyscan, enable_bitmapscan and enable_seqscan to on.

Comments

The six complete plans appear immediately before this comparison. Costs are planner units; measured times are milliseconds.

Screenshot Q4.1 (1)	Marks: /2

PostgreSQL execution transcript Q4.1 step 1 | connection 1 | part 1/1

```
-- connection 1
BEGIN;
BEGIN
SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 row)

SELECT v.vehicle_id, latest.event_id,
       latest.event_type, latest.event_time
FROM VEHICLES v
CROSS JOIN LATERAL (
    SELECT e.event_id, e.event_type, e.event_time
    FROM BOOKING_EVENTS e
    WHERE e.vehicle_id = v.vehicle_id
    ORDER BY e.event_time DESC, e.event_id DESC
    LIMIT 1
) latest
WHERE v.vehicle_id = 'VEH-0001'
  AND latest.event_type = 'Return';
 vehicle_id | event_id | event_type |     event_time
------------+----------+------------+---------------------
 VEH-0001   |   194486 | Return     | 2026-06-27 11:05:11
(1 row)

-- Keep this transaction open. Run 4.1.2 in connection 2.
```

Source: 4.1.1.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Connection 1 begins a READ COMMITTED transaction and returns VEH-0001 with latest event 194486, a Return at 2026-06-27 11:05:11. The lateral subquery first selects the latest event across all event types, using event_id to break timestamp ties. Only after that does the outer query test for Return. An older Return cannot incorrectly qualify a vehicle whose latest state is Reserve, Collect or Incident.

The selected vehicle is kept fixed for the same availability check in connection 2. Connection 1 stays open after this step.

Screenshot Q4.1 (2)	Marks: /0.5

PostgreSQL execution transcript Q4.1 step 2 | connection 2 | part 1/1

```
-- connection 2
BEGIN;
BEGIN
SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 row)

SELECT v.vehicle_id, latest.event_id,
       latest.event_type, latest.event_time
FROM VEHICLES v
CROSS JOIN LATERAL (
    SELECT e.event_id, e.event_type, e.event_time
    FROM BOOKING_EVENTS e
    WHERE e.vehicle_id = v.vehicle_id
    ORDER BY e.event_time DESC, e.event_id DESC
    LIMIT 1
) latest
WHERE v.vehicle_id = 'VEH-0001'
  AND latest.event_type = 'Return';
 vehicle_id | event_id | event_type |     event_time
------------+----------+------------+---------------------
 VEH-0001   |   194486 | Return     | 2026-06-27 11:05:11
(1 row)

INSERT INTO BOOKING_EVENTS
    (member_id, vehicle_id, event_type, event_time, amount)
VALUES (2, 'VEH-0001', 'Reserve', NOW() + INTERVAL '3 days', NULL);
INSERT 0 1
COMMIT;
COMMIT
```

Source: 4.1.2.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Connection 2 also reports READ COMMITTED and sees the same latest Return while connection 1 is open. It inserts a Reserve for member 2 at NOW() + INTERVAL '3 days' and commits successfully. The plain availability SELECT in connection 1 did not lock the vehicle against this insertion.

Screenshot Q4.1 (3)	Marks: /1

PostgreSQL execution transcript Q4.1 step 3 | connection 1 | part 1/1

```
-- connection 1, after connection 2 has committed
INSERT INTO BOOKING_EVENTS
    (member_id, vehicle_id, event_type, event_time, amount)
VALUES (1, 'VEH-0001', 'Collect', NOW(), NULL);
ERROR:  Vehicle VEH-0001 is reserved by member 2
CONTEXT:  PL/pgSQL function udf_bi_collect() line 22 at RAISE
-- The trigger raises an exception, so COMMIT ends this failed transaction as ROLLBACK.
COMMIT;
ROLLBACK
SELECT event_id, member_id, vehicle_id, event_type, event_time, amount
FROM BOOKING_EVENTS
WHERE vehicle_id = 'VEH-0001'
ORDER BY event_time DESC, event_id DESC LIMIT 3;
 event_id | member_id | vehicle_id | event_type |        event_time          | amount
----------+-----------+------------+------------+----------------------------+--------
   196442 |         2 | VEH-0001   | Reserve    | 2026-10-29 02:58:03.71202 |
   194486 |     20450 | VEH-0001   | Return     | 2026-06-27 11:05:11       | 115.49
   194367 |     20450 | VEH-0001   | Collect    | 2026-06-27 04:21:33       |
(3 rows)
```

Source: 4.1.3.txt | Actual psql output | Alignment whitespace reflowed for readability

Comments

Connection 1 attempts a present-time Collect for member 1 after connection 2 commits. BI_COLLECT rejects it with ERROR: Vehicle VEH-0001 is reserved by member 2. The transaction is then failed, so the attempted COMMIT produces ROLLBACK. No new Collect is stored.

The verification query shows Reserve event 196442 as the latest event, ahead of historical Return 194486 and historical Collect 194367. Of the two competing new events in this experiment, only the Reserve survives. The Incident from Q1.3 remains a separate older historical event. Sequence allocation can leave a gap for the rejected Collect; a consumed serial value does not prove that a row committed.

Screenshot Q4.1 (4)	Marks: /2.5

The observed schedule is T1 BEGIN -> T1 availability read -> T2 BEGIN -> T2 availability read -> T2 Reserve insert -> T2 COMMIT -> T1 Collect attempt -> trigger exception -> T1 ROLLBACK.

At READ COMMITTED each statement gets a fresh snapshot. T1's initial SELECT sees the previously committed Return. The Collect INSERT runs after T2 commits, so the trigger's later query can see T2's Reserve. Sorting by event_time DESC and event_id DESC makes that future-dated Reserve the latest event, and its member differs from member 1. The initial availability answer has gone stale by the time T1 attempts the Collect.

An ordinary SELECT takes an ACCESS SHARE relation lock, which is compatible with the other transaction's insertion; it takes no row lock that reserves this vehicle. MVCC therefore allows T2 to commit while T1 is still open. In this observed schedule neither connection waits on a conflicting row lock: T1's Collect is rejected by a business-rule exception. This is not a deadlock.

For the co-operative, the member at the bay loses access even though the other member booked three days ahead. The trigger treats the most recent timestamped event as the current state without distinguishing a future reservation interval from present availability. A valid future reservation therefore becomes an immediate hold under this simplified model. Serializing vehicle operations could close a check-then-write race, but interval-aware reservation rules are also needed to support a car being available now and reserved later.

Comments

This explanation describes the supplied trigger and the actual schedule; a snapshot is a consistent read, not a reservation of the vehicle.

Screenshot Q4.2	Marks: /3

The SQL script labels both connections and every statement in the exact order sent. Both connections begin. T1 updates VEH-0401 by +10; T2 updates VEH-0501 by +20. T1 then requests VEH-0501 and waits. Before the deadlock timeout, T2 requests VEH-0401, completing the cycle. The complete server error and the COMMIT/ROLLBACK outcomes are shown below.

PostgreSQL execution transcript Q4.2 | initial locks | both connections | part 1/1

```
-- Step 1 | connection 1
BEGIN;
BEGIN

-- Step 2 | connection 2
BEGIN;
BEGIN

-- Step 3 | connection 1
UPDATE VEHICLES SET odometer = odometer + 10 WHERE vehicle_id = 'VEH-0401';
UPDATE 1

-- Step 4 | connection 2
UPDATE VEHICLES SET odometer = odometer + 20 WHERE vehicle_id = 'VEH-0501';
UPDATE 1
```

Source: deadlock-before.txt | Actual psql output | Alignment whitespace reflowed for readability

PostgreSQL execution transcript Q4.2 | cycle and resolution | both connections | part 1/1

```
-- Step 5 | connection 1
UPDATE VEHICLES SET odometer = odometer + 10 WHERE vehicle_id = 'VEH-0501';
ERROR:  deadlock detected
DETAIL:  Process 50868 waits for ShareLock on transaction 797; blocked by process 46904.
Process 46904 waits for ShareLock on transaction 796; blocked by process 50868.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (3,93) in relation "vehicles"

-- Step 6 | connection 2
UPDATE VEHICLES SET odometer = odometer + 20 WHERE vehicle_id = 'VEH-0401';
UPDATE 1

-- Step 7 | connection 1
COMMIT;
ROLLBACK

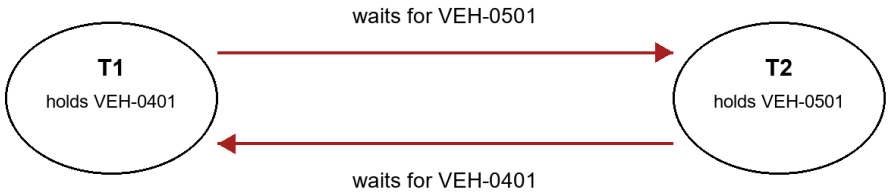
-- Step 8 | connection 2
COMMIT;
COMMIT

-- Step 9 | connection 1
SELECT vehicle_id, odometer FROM VEHICLES WHERE vehicle_id IN ('VEH-0401', 'VEH-0501')
  ORDER BY vehicle_id;
 vehicle_id | odometer
------------+----------
 VEH-0401   |   149120
 VEH-0501   |   122170
(2 rows)
```

Source: deadlock-cycle.txt | Actual psql output | Alignment whitespace reflowed for readability

Screenshot Q4.2	Marks: /3

WAIT FOR GRAPH BEFORE DEADLOCK DETECTION



Cycle marked in red T1 -> T2 -> T1

Comments

Immediately before intervention, T1 holds the row lock for VEH-0401 and waits for the row held by T2 on VEH-0501; T2 holds VEH-0501 and waits for VEH-0401. The wait-for edges are T1 -> T2 and T2 -> T1, which form a cycle. The ShareLock-on-transaction wording in PostgreSQL's error reflects waiting for the transaction owning the conflicting row version to finish.

In this run PostgreSQL selected T1 as the deadlock victim and aborted its transaction. T1's first +10 update was rolled back along with its second attempt, and its COMMIT reported ROLLBACK. Releasing T1's locks allowed T2's blocked update to finish; T2 then committed both +20 corrections. The final odometers are 149,120 for VEH-0401 and 122,170 for VEH-0501. The victim choice can differ in a rerun; applications must be prepared to retry the aborted transaction.

A consistent lock order prevents this cycle: every correction transaction must lock or update the vehicles in ascending vehicle_id order, so both request VEH-0401 before VEH-0501. The co-operative pays for waiting/queuing when staff edit overlapping vehicles and must enforce the ordering rule across all code paths; unrelated vehicles can still be updated concurrently.

Screenshot P	Marks: /2

The report follows the supplied submission template. All required SQL statements are included in the output screenshots; the 20 separately named scripts contain the executable code. All query plans are shown in full across numbered screenshot parts. Ordinary query results with more than 20 rows show only the first ten and last ten rows, with an explicit omission marker. No displayed result tuple wraps across lines.

The evidence was produced by real psql execution against PostgreSQL 18.6 on Windows in an isolated local database, using UTF-8, C collation, Australia/Brisbane time and default READ COMMITTED isolation. The assignment links PostgreSQL 17 documentation; optimizer details and measurements can vary with version, statistics, cache state and hardware. The provided setup SQL was loaded once before running the scripts in question order; the scripts do not repeat dataset setup.

The screenshots capture a local viewer of the saved psql transcripts rather than a native terminal window. Alignment whitespace and plan indentation are compacted for readability; output values and error text are unchanged. Full original transcripts and image files are retained in evidence. Q4 was executed with two separate persistent psql connections and commands interleaved in the reported order.

Reported cost values are planner estimates in arbitrary units, not milliseconds. Execution Time is the measured server time for EXPLAIN ANALYZE, not a repeated benchmark or client transfer time. Parent-node times include child work, and parallel actual-row figures must be interpreted with their loop counts.

Documentation checked: PostgreSQL 18 Query Planning (https://www.postgresql.org/docs/18/runtime-config-query.html), Transaction Isolation (https://www.postgresql.org/docs/18/transaction-iso.html), and Explicit Locking (https://www.postgresql.org/docs/18/explicit-locking.html). The supplied assignment, dataset and course materials are the primary task sources.

Comments

The penalty and marking fields below are left blank for the teaching team.

Penalty: _____

Reason of the Penalty

(Late Submission/No Show to Tutor/Purely AI Generated Solutions): _____

Final Marks: _____ / 30

---ooo000OOO000ooo---